

SV9 - Digital Verification using SV & UVM

=====Assignment (2)=====

Data Types & Constrained Random Assignment

Name
Ibrahim Mohamed Mohamed Hanafy

Question1: Dynamic Arrays

-Requirements:

- 1) Write a module to test dynamic array data type and its predefined methods. Run Questasim to make sure the display statements are working as expected.
 - declare two dynamic arrays dyn_arr1, dyn_arr2 of type int
 - initialize dyn_arr2 array elements with (9,1,8,3,4,4)
 - allocate six elements in array dyn_arr1 using new operator
 - initialize array dyn_arr1 with index as its value using foreach
 - display dyn_arr1 and its size
 - Expected output: (0,1,2,3,4,5), 6
 - delete array dyn_arr1
 - reverse, sort, reverse sort and shuffle the array dyn_arr2 and display dyn_arr2 after using each method
 - Expected output: (4,4,3,8,1,9), (1,3,4,4,8,9), (9,8,4,4,3,1), <shuffled_array>

-Module:

```
module testing_dynamic_arrays;
    initial begin

        int dyn_arr1[];
        int static_initialization[6] = '{9,1,8,3,4,4}';
        int dyn_arr2[];
        dyn_arr2 = new[6](static_initialization);

        dyn_arr1 = new[6];

        foreach (dyn_arr1[i]) begin
            dyn_arr1[i] = i;
        end

        $display("dyn_arr1 elements: %p | Size = %d", dyn_arr1, $size(dyn_arr1));

        dyn_arr1.delete();

        dyn_arr2.reverse();
        $display("dyn_arr2 elements reversed: %p", dyn_arr2);

        dyn_arr2.sort();
        $display("dyn_arr2 elements sorted: %p", dyn_arr2);

        dyn_arr2.rsort();
        $display("dyn_arr2 elements reverse sorted: %p", dyn_arr2);

        dyn_arr2.shuffle();
        $display("dyn_arr2 elements shuffled: %p", dyn_arr2);
    end
endmodule
```

-Results:

```
# Loading workspace_dynamic_arrays(1888,  
# dyn_arr1 elements: '{0, 1, 2, 3, 4, 5}' | Size = 6  
# dyn_arr2 elements reversed: '{4, 4, 3, 8, 1, 9}'  
# dyn_arr2 elements sorted: '{1, 3, 4, 4, 8, 9}'  
# dyn_arr2 elements reverse sorted: '{9, 8, 4, 4, 3, 1}'  
# dyn_arr2 elements shuffled: '{4, 1, 8, 3, 9, 4}'
```

Question 2: Counter

-Counter Design Code:

```
8  module counter (clk ,rst_n, load_n, up_down, ce, data_load, count_out)
20  //optional: missing validation of allowed widths
21
22  always @(posedge clk) begin
23      if (!rst_n)
24          count_out <= 0;
25      else if (!load_n)
26          count_out <= data_load;
27      else if (ce)
28          if (up_down)
29              count_out <= count_out + 1;
30          else
31              count_out <= count_out - 1;
32      //missing else statement infers a latch
33      //which is actually the needed implementation
34  end
35
36  assign max_count = (count_out == {WIDTH{1'b1}})? 1:0;
37  assign zero = (count_out == 0)? 1:0;
38
39  endmodule
```

-No Bugs found

-Verification Plan:

Label	Design Requirement Description	Stimulus Generation	Functionality Check
Reset	When the reset is asserted, the output counter value should be low	Directed at the start of the simulation, then randomized such that it is off most of the time	A checker in the testbench to make sure the output is correct Can also check for zero flag
Load	When the load is asserted, the output count_out should take the value of the load_data input	Randomization, ST: It is active 70% of the time	A checker in the testbench to make sure the output is correct, matching the input
Enable	When enabled, the counter counts up/down	Randomization, ST: It is active 70% of the time	A checker to make the ensure counting every cycle

- Package:

```
package class_pkg;

    class RandomControl;
        rand logic rst_n, load_n, ce;

        constraint Reset { //off for 90% of the time
            rst_n dist {1:/90 , 0:/10};
        }
        constraint Load { //on for 70% of the time
            load_n dist {0:/70 , 1:/30};
        }
        constraint Enable { //on for 70% of the time
            ce dist {1:/70 , 0:/30};
        }
    endclass

endpackage
```

- Testbench:

```
counter #(.WIDTH(WIDTH)) uut (.*);
```

```
initial begin
    if (!(WIDTH inside {4,6,8})) begin
        $error("Invalid WIDTH=%0d. Allowed values are 4, 6, or 8.", WIDTH);
        $fatal;
    end
end
```

```
integer correct_count = 0;
integer error_count = 0;
```

```
initial begin
    RandomControl cntrl;
    cntrl = new;
    correct_count = 0; error_count = 0;
    expected_count_out = 0; expected_max_count = 0; expected_zero = 1;
    $display("=====TestBench Start=====");
```

```
    //Test Series 1: Assert reset
    assert_reset();
```

```
    //Test Series 2: Randomization loop
    repeat(1000) begin //take care when repeating for sequential testbenches
        assert(cntrl.randomize()); // as we check and stimulate after
    //use object's values
        rst_n = cntrl.rst_n;
        load_n = cntrl.load_n;
        ce = cntrl.ce;
    //fill other inputs
        up_down = $random;
        data_load = $random;
```

```
        compute_expected(); //Golden model
```

```
        @(negedge clk); //cant check at n
        check_result();
        #1;
```

```
    end
```

```
    //Test Series 3: Assert reset again
    assert_reset();
```

```
task assert_reset;
    @(negedge clk)
    rst_n = 0;
    @(negedge clk);
    if (count_out != 0) begin
        error_count = error_count + 1;
        $display("Error Resetting");
    end
    else
        correct_count = correct_count + 1;
    @(negedge clk);
    rst_n = 1;
endtask
```

```
$display("%t: At end of test error count is %0d and correct count = %0d", $time,
$display("=====TestBench End=====");
```

```

task compute_expected; //Golden Model
    if (!rst_n)
        expected_count_out = {WIDTH{1'b0}};
    else if (!load_n)
        expected_count_out = data_load;
    else if (ce) begin
        if (up_down)
            expected_count_out = expected_count_out + 1'b1;
        else
            expected_count_out = expected_count_out - 1'b1;
    end
    else
        expected_count_out = expected_count_out; // hold value

    expected_max_count = (expected_count_out == {WIDTH{1'b1}});
    expected_zero      = (expected_count_out == {WIDTH{1'b0}});

endtask

```

```

task check_result;

    if ((count_out != expected_count_out) ||
        (max_count != expected_max_count) ||
        (zero      != expected_zero)) begin
        error_count = error_count + 1;
        $display("=====");
        $display("Error: rst_n = %b | load_n = %b | ce = %b | up_down = %b" , rst_n,
        $display("Data Load = %d" , data_load);
        $display("Expected count = %d | Actual = %d", expected_count_out, count_out);
        $display("Expected max = %b | Actual = %b || Expected zero = %b | Actual = %b",
    end
    else
        correct_count = correct_count + 1;

endtask

```

- Do file:

```

#vlib work
vlog -sv ../../../../VS_code/Session2/Assignment2/Q2_Counter/* +cover -covercells
vsim -voptargs=+acc work.Counter_tb -cover
#add wave *
coverage save Counter_tb.ucdb -onexit
#do wave.do
run -all

vcover report Counter_tb.ucdb -details -all -output code_cvr.txt

```

-Coverage Report:

```
=====
=== File: ../../VS_code/Session2/Assignment2/Q2_Counter/counter.v
=====

Statement Coverage:
  Enabled Coverage      Active      Hits      Misses % Covered
  -----
  Stmts                7         7         0     100.0

Branch Coverage:
  Enabled Coverage      Active      Hits      Misses % Covered
  -----
  Branches            10         10         0     100.0

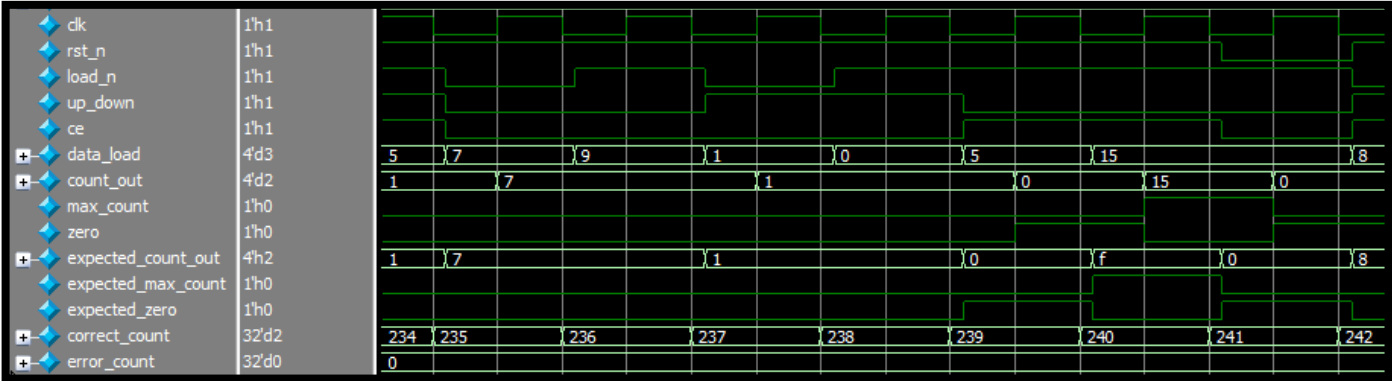
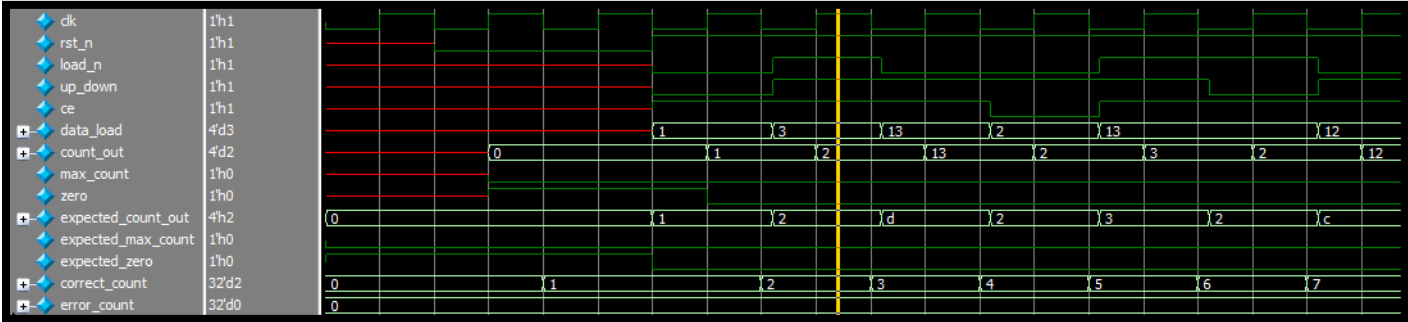
Toggle Coverage:
  Enabled Coverage      Active      Hits      Misses % Covered
  -----
  Toggle Bins         30         30         0     100.0

=====Toggle Details=====

Toggle Coverage for File ../../VS_code/Session2/Assignment2/Q2_Counter/counter.v --

  Line                Node      1H->0L      0L->1H  "Coverage"
  -----
   10                 clk         1          1     100.00
   11                 rst_n         1          1     100.00
   12                 load_n         1          1     100.00
   13                 up_down         1          1     100.00
   14                  ce         1          1     100.00
   15      data_load[3]         1          1     100.00
   15      data_load[2]         1          1     100.00
   15      data_load[1]         1          1     100.00
   15      data_load[0]         1          1     100.00
   17      count_out[3]         1          1     100.00
   17      count_out[2]         1          1     100.00
   17      count_out[1]         1          1     100.00
   17      count_out[0]         1          1     100.00
   18          max_count         1          1     100.00
   19              zero         1          1     100.00
```

-Waveforms:



Design 3: ALSU

-ALSU Design Code:

```
parameter INPUT_PRIORITY = "A";
parameter FULL_ADDER = "ON"; //to allow use of cin

input clk, cin, rst, red_op_A, red_op_B, bypass_A, bypass_B, direction;
input [2:0] opcode;
input signed [2:0] A, B;

output reg [15:0] leds;
output reg signed [5:0] out;

reg red_op_A_reg, red_op_B_reg, bypass_A_reg, bypass_B_reg, direction_reg;
reg signed [1:0] cin_reg; //cin is 1 bit<=====
reg [2:0] opcode_reg;
reg signed [2:0] A_reg, B_reg;

wire invalid_red_op, invalid_opcode, invalid;

//Invalid handling=====
assign invalid_red_op = (red_op_A_reg | red_op_B_reg) & (opcode_reg[0] & opcode_reg[1]);
assign invalid_opcode = opcode_reg[1] & opcode_reg[2];
assign invalid = invalid_red_op | invalid_opcode;

always @(posedge clk or posedge rst) begin
    if(rst) begin
        cin_reg <= 0;
        red_op_B_reg <= 0;
        red_op_A_reg <= 0;
        bypass_B_reg <= 0;
        bypass_A_reg <= 0;
        direction_reg <= 0;
        serial_in_reg <= 0;
        opcode_reg <= 0;
        A_reg <= 0;
        B_reg <= 0;
    end else begin
        cin_reg <= cin;
        red_op_B_reg <= red_op_B;
        red_op_A_reg <= red_op_A;
        bypass_B_reg <= bypass_B;
        bypass_A_reg <= bypass_A;
        direction_reg <= direction;
        serial_in_reg <= serial_in;
        opcode_reg <= opcode;
        A_reg <= A;
        B_reg <= B;
    end
end

//leds output blinking=====
always @(posedge clk or posedge rst) begin
    if(rst) begin
        leds <= 0;
    end else begin
        if (invalid)
            leds <= ~leds;
        else
            leds <= 0;
    end
end
```

```

always @(posedge clk or posedge rst) begin
    if(rst) begin
        out <= 0;
    end
    else begin
        if (bypass_A_reg && bypass_B_reg)
            out <= (INPUT_PRIORITY == "A")? A_reg: B_reg;
        else if (bypass_A_reg)
            out <= A_reg;
        else if (bypass_B_reg)
            out <= B_reg;
        else if (invalid)
            out <= 0;
        else begin
            case (opcode) //Should use the registered opcode here<====
                3'h0: begin //OR
                    if (red_op_A_reg && red_op_B_reg)
                        out <= (INPUT_PRIORITY == "A")? |A_reg: |B_reg;
                    else if (red_op_A_reg)
                        out <= |A_reg;
                    else if (red_op_B_reg)
                        out <= |B_reg;
                    else
                        out <= A_reg | B_reg;
                end
                3'h1: begin //XOR
                    if (red_op_A_reg && red_op_B_reg)
                        out <= (INPUT_PRIORITY == "A")? ^A_reg: ^B_reg;
                    else if (red_op_A_reg)
                        out <= ^A_reg;
                    else if (red_op_B_reg)
                        out <= ^B_reg;
                    else
                        out <= A_reg ^ B_reg;
                end
                3'h2: out <= A_reg + B_reg; // cin wasnt taken into considerat
                3'h3: out <= A_reg * B_reg;
                3'h4: begin //SHIFT
                    if (direction_reg)
                        out <= {out[4:0], serial_in_reg};
                    else
                        out <= {serial_in_reg, out[5:1]};
                end
                3'h5: begin //ROTATE
                    if (direction_reg)
                        out <= {out[4:0], out[5]};
                    else
                        out <= {out[0], out[5:1]};
                end //3'h6 3'h7 should have been included<====
            endcase
        end
    end
end

```

-Bugs & Fixes

- Adder didn't implement cin where we check if FULL_ADDER is ON
- Registered opcode wasn't used in case block
- cin_reg size is wrong
- cases 3'h6 & 3'h7 weren't stated

-Verification Plan:

Label	Design Requirement Description	Stimulus Generation	Functionality Check
Reset	When the reset is asserted, the output counter value should be low	Directed at the start of the simulation, Then randomized in a loop and constrained to be off most of the time	A checker in the testbench to make sure the output is zero
Addition	If not Invalid: Addition should be correct with availability of carry cin if FULL_ADDER is ON	Randomization while constrained inputs : MAXPOS MAXNEG ZERO most of the time	verified against the golden model
OR / XOR	When their opcode is set: if reduction set: According to priority preform reduction operation elsewise preform bitwise operation in i/p(s)	Randomized in a loop and constrained such that for reduction: the selected i/p has one bit high and the unselected one to have all low bits	verified against the golden model reduction should now always yield 1
Addition	If not Invalid: Addition should be correct with availability of carry cin if FULL_ADDER is ON	Randomization while constrained inputs : MAXPOS MAXNEG ZERO most of the time	verified against the golden model
Validity	-Opcode bits are set to 110 or 111 -red_op_A or red_op_B are high and the opcode is not OR or XOR operation Both yield invalid	Constrained such that invalid/ illegal cases are less frequent than valid ones	LED blinking functionality
bypass	The i/p is simply passed as o/p	Constrained such that bypasses aren't frequent	Check o/p vs i/p

-Package:

```
package class_pkg;

parameter INPUT_PRIORITY = "A";
parameter FULL_ADDER = "ON";

typedef enum reg [2:0] {OR, XOR, ADD, MULT, SHIFT, ROTATE, INVALID_6, INVALID_7} opcode_e;
localparam MAXPOS = 3, ZERO = 0, MAXNEG = -4;

class RandomControl;
    rand logic rst,red_op_A, red_op_B, bypass_A, bypass_B, cin, direction, serial_in;
    rand logic [2:0] opcode;
    rand logic signed [2:0] A, B;

    constraint Reset { //off for 90% of the time
        rst dist {0:/90 , 1:/10};
    }

    constraint Inputs { //MAXs and Zero 80% of the time for ADD and MULT
        if((opcode == ADD) | (opcode == MULT)){
            A dist {MAXPOS:=40,
                MAXNEG:=40,
                ZERO:=40,
                [-3:-1], [1:2] := 15};
            B dist {MAXPOS:=40,
                MAXNEG:=40,
                ZERO:=40,
                [-3:-1], [1:2] := 15}; //15/135
        }
    }

    constraint Bypass { //off for 90% of the time
        bypass_A dist {0:/90 , 1:/10};
        bypass_B dist {0:/90 , 1:/10};
    }

    constraint Validity { //Invalid for 10% of the time
        opcode dist {INVALID_6:/10,INVALID_7:/10,
            [OR:ROTATE]:/80};
        if(opcode == (OR | XOR)){
            red_op_A dist{0:/50 , 1:/50}; //both red high is unlikely?
            if(red_op_A){
                red_op_B dist{0:/80 , 1:/20};
            }
        }
    }

    constraint Reduction_Operations { //selected has one bit set
        if(opcode inside {OR, XOR}){ //unslected has all low
            if(red_op_A){
                A dist { 4:=80, 1:=80, 2:=80, [-3:0]:=20, 3:=20 };
                B == 3'b000;
            }
            else if(red_op_B){
                B dist { 4:=80, 1:=80, 2:=80, [-3:0]:=20, 3:=20 };
                A == 3'b000;
            }
        }
    }
endclass
```

- Golden Model:

```
// Reference output model
always @(posedge clk or posedge rst) begin
    if (rst) begin
        out <= 6'sd0;
    end
    else begin
        if (bypass_A_reg && bypass_B_reg)
            out <= (INPUT_PRIORITY == "A") ? {{3{A_reg}}}
        else if (bypass_A_reg)
            out <= {{3{A_reg[2]}}, A_reg};
        else if (bypass_B_reg)
            out <= {{3{B_reg[2]}}, B_reg};
        else if (invalid)
            out <= 6'sd0;
        else begin
            case (opcode_reg)
                3'h0: begin // OR
                    if (red_op_A_reg && red_op_B_reg)
                        out <= (INPUT_PRIORITY == "A") ? {{5{1'b0}}, |A_reg} : {{5{1'b0}}, |B_reg};
                    else if (red_op_A_reg)
                        out <= {{5{1'b0}}, |A_reg};
                    else if (red_op_B_reg)
                        out <= {{5{1'b0}}, |B_reg};
                    else
                        out <= A_reg | B_reg;
                end

                3'h1: begin // XOR
                    if (red_op_A_reg && red_op_B_reg)
                        out <= (INPUT_PRIORITY == "A") ? {{5{1'b0}}, ^A_reg} : {{5{1'b0}}, ^B_reg};
                    else if (red_op_A_reg)
                        out <= {{5{1'b0}}, ^A_reg};
                    else if (red_op_B_reg)
                        out <= {{5{1'b0}}, ^B_reg};
                    else
                        out <= A_reg ^ B_reg;
                end

                3'h2: begin // ADD
                    if (FULL_ADDER == "ON")
                        out <= A_reg + B_reg + cin_reg;
                    else
                        out <= A_reg + B_reg;
                end

                3'h3: begin // MULT
                    out <= $signed({{3{A_reg[2]}}, A_reg}) *
                        $signed({{3{B_reg[2]}}, B_reg});
                end

                3'h4: begin // SHIFT
                    if (direction_reg)
                        out <= {out[4:0], serial_in_reg};
                    else
                        out <= {serial_in_reg, out[5:1]};
                end

                3'h5: begin // ROTATE
                    if (direction_reg)
                        out <= {out[4:0], out[5]};
                    else
                        out <= {out[0], out[5:1]};
                end

                3'h6, 3'h7: out <= 6'sd0;
            endcase
        end
    end
end

// LEDs blink on invalid
always @(posedge clk or posedge rst) begin
    if(rst) begin
        leds <= 0;
    end else begin
        if (invalid)
            leds <= ~leds;
        else
            leds <= 0;
        end
    end
end

// Register all inputs except clk/rst
always @(posedge clk or posedge rst) begin
    if (rst) begin
        cin_reg <= 1'b0;
        red_op_A_reg <= 1'b0;
        red_op_B_reg <= 1'b0;
        bypass_A_reg <= 1'b0;
        bypass_B_reg <= 1'b0;
        direction_reg <= 1'b0;
        serial_in_reg <= 1'b0;
        opcode_reg <= 3'b000;
        A_reg <= 3'sd0;
        B_reg <= 3'sd0;
    end
    else begin
        cin_reg <= cin;
        red_op_A_reg <= red_op_A;
        red_op_B_reg <= red_op_B;
        bypass_A_reg <= bypass_A;
        bypass_B_reg <= bypass_B;
        direction_reg <= direction;
        serial_in_reg <= serial_in;
        opcode_reg <= opcode;
        A_reg <= A;
        B_reg <= B;
    end
end
```

- Testbench:

```
initial begin
    RandomControl cntrl;
    cntrl = new;
    correct_count = 0; error_count = 0;
    expected_out = 0; expected_leds = 0;
    $display("=====TestBench Start=====");

    //Test Series 1: Assert reset
    assert_reset();

    //Test Series 2: Randomization loop
    repeat(1000) begin //take care when repeating for sequential testbenches
        assert(cntrl.randomize()); // as we check and stimulate after 1 tick
    //use object's values
        A = cntrl.A;
        B = cntrl.B;
        opcode = opcode_e'(cntrl.opcode);

        cin = cntrl.cin; rst = cntrl.rst ;
        red_op_A=cntrl.red_op_A; red_op_B = cntrl.red_op_B;
        bypass_A = cntrl.bypass_A; bypass_B = cntrl.bypass_B;
        direction= cntrl.direction; serial_in = cntrl.serial_in;

        //wait for two clock cycles
        @(negedge clk);
        @(negedge clk);

        check_result();
        #1;
    end

    //Test Series 3: Assert reset again
    assert_reset();

    $display("%t: At end of test error count is %0d and correct count = %0d", $time, error_count, correct_count);
    $display("=====TestBench End=====");
    $stop;
end
```

```
ALSU #(
    .INPUT_PRIORITY(INPUT_PRIORITY),
    .FULL_ADDER(FULL_ADDER)
) uut (.*);
ALSU_golden #(
    .INPUT_PRIORITY(INPUT_PRIORITY),
    .FULL_ADDER(FULL_ADDER)
) golden (.out(expected_out),.leds(expected_leds),.*);
```

```
task check_result;

    if((out == expected_out) && (leds == expected_leds))
        correct_count = correct_count + 1;
    else
        error_count = error_count + 1;

    if (out != expected_out) begin
        $display("=====");
        $display("Error: opcode = %s | bypass_A = %b | bypass_B = %b | red_op_A = %b | red_op_B = %b | cin = %b" , opcode.name,
        $display("direction = %b | serial_in = %b" , direction,serial_in);
        $display("A: %d | B: %d", A, B);
        $display("Expected out = %d | Actual = %d", out, expected_out);
    end

    if (leds != expected_leds) begin
        $display("=====");
        $display("Error: opcode = %s | bypass_A = %b | bypass_B = %b | red_op_A = %b | red_op_B = %b | cin = %b" , opcode.name,
        $display("direction = %b | serial_in = %b" , direction,serial_in);
        $display("A: %d | B: %d", A, B);
        $display("Expected leds = %d | Actual = %d", leds, expected_leds);
    end
endtask
```

- Do file

```
#vlib work
vlog -sv ../../../../VS_code/Session2/Assignment2/Q3_ALSU/* +cover -covercells
vsim -voptargs=+acc work.ALSU_tb -cover
add wave *
coverage save ALSU_tb.ucdb -onexit
do wave.do
run -all

vcover report ALSU_tb.ucdb -details -all -output code_cvr.txt
```

Coverage Report

=====				
=== File: ../../../../VS_code/Session2/Assignment2/Q3_ALSU/ALSU.v				
=====				
Statement Coverage:				
Enabled Coverage	Active	Hits	Misses	% Covered
-----	-----	----	-----	-----
Stmts	51	49	2	96.0
Branch Coverage:				
Enabled Coverage	Active	Hits	Misses	% Covered
-----	-----	----	-----	-----
Branches	34	32	2	94.1
Toggle Coverage:				
Enabled Coverage	Active	Hits	Misses	% Covered
-----	-----	----	-----	-----
Toggle Bins	118	118	0	100.0

-The 2 missed statements and branches are the ones concerning invalid opcodes in the case, they are infact impossible to reach as we check for invalidity **before** the case

-Waveforms:

